

Start with easy tasks to show you can read in a file, create some variables and data structures, and manipulate their contents. Then move onto something more interesting.

Look at the data that we provided with this assessment and think of something interesting to do with it using whatever libraries you like. Describe what you decide to do with the data and why it might be interesting or useful to the company to do it.

You can add additional data if you need to - either download it or access it using `requests`. Produce working code to implement your ideas in as many cells as you need below. There is no single right answer, the aim is to simply show you are competent in using python for data analysis. Exactly how you do that is up to you.

For a distinction class grade, this must show originality, creative thinking, and insights beyond what you've been taught directly on the module.

Structure

You may structure the project how you wish, but here is a suggested guideline to help you organise your work:

1. Data Exploration - Explore the data and show you understand its structure and relations
2. Data Validation - Check the quality of the data. Is it complete? Are there obvious errors?
3. Data Visualisation - Gain an overall understanding of the data with visualisations
4. Data Analysis = Set some questions and use the data to answer them
5. Data Augmentation - Add new data from another source to bring new insights to the data you already have

Remember to make sure you are working completely on your own.

Don't work in a group or with a friend

You may NOT use any automated code generation or analytics tools for this assignment, so do not use tools like GPT. You can look up the syntax for the functions you use, but you must write the code yourself and the comments must provide an insightful analysis of the results.

Data Exploration

Text Narration of Data Exploration

Imports necessary libraries, including pandas, os, csv, json, matplotlib.pyplot, seaborn, and numpy. Defines a function named Load_csv to load and explore CSV files. This function takes a file name as input, reads the CSV file, and returns a DataFrame. Calls Load_csv function to load three CSV files: 'products.csv', 'reviews.csv', and 'users.csv'. Displays information, statistics, and the first few rows of each CSV file to understand its data structure. Defines a function named Load_json_files to load JSON files. This function reads a JSON file using the read_json function, replaces empty strings with NaN, checks for empty values, and returns a DataFrame. Defines a function named remove_negative_and_null_values to replace negative values with NaN in numerical columns and drops rows with NaN values. Calls Load_json_files function to load two JSON files: 'jcpenney_products.json' and 'jcpenney_reviewers.json'. Displays information and statistics for each JSON file to understand its data structure. Displays the first few rows of each JSON file.

In [101...

```
#Importing necessary Liabrararies.

import pandas as pd
import os
import csv
import json
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

#Defined function named Load_csv to load and explore csv files.
def Load_csv(File_name):

    # Get the current working directory and store it in path variable.
    path = os.getcwd()

    #To get the full path joining current working directory with provided file name.
    files_csv = os.path.join(path, File_name)

    #Read the csv file and storing it in DataFrame.
    data_file = pd.read_csv(files_csv)

    #Return the DataFrame
    return data_file

print("_____")
print("\nProduct.csv Table Description")
print("_____")

#Calling load_csv function to load the file into dataframe.Creating products_csv variable
#file received in repsonse of the function.
products_csv=Load_csv('products.csv')

#To print information about products.csv to understand data structure.
print(products_csv.info())

print("_____")
print("\nProduct.csv Table Statistics")
print("_____")
```

```

# display product.csv statistics for data understanding. It will provide us mean,minim
#price and Av_score present in product.csv table.
product = products_csv.describe()
display(product)

print("\n_____")
print("\nProduct Table First Few Rows")
print("_____")

#Print first few rows of products dataframe to undertsand the data structure.
display(products_csv.head())

print("\n\n")
print("_____")
print("\nReviews.csv Table Description")
print("_____ \n")

#Calling load_csv function to load the file into dataframe and store response in review
reviews_csv=Load_csv('reviews.csv')

#To print information about reviews.csv to understand data structure.
print(reviews_csv.info())

print("\n")
print("_____")
print("\nReviews.csv Table Statistics")
print("_____ \n")

# display reviews_csv statistics for data understanding.It will provide us mean,minimu
#score present in reviews.csv table.
review = reviews_csv.describe()
display(review)

print("\n_____")
print("\nReviews Table First Few Rows")
print("_____")

#Print first few rows of reviews dataframe to undertsand the data structure.
display(reviews_csv.head())

print("\n\n")
print("_____")
print("\nUsers.csv Table Description")
print("_____ \n")

#Calling load_csv function to load the file into dataframe and store response in users
users_csv=Load_csv('users.csv')

#To print information about users.csv to understand data structure.
print(users_csv.info())

print("\n")
print("_____")

```

```

print("\nUsers.csv Table Statistics")
print("_____ \n")

#Display user.csv statistics for data understanding.It will provide us mean,minimum,maximum,
#DOB,Username and State present in Users.csv table.
user = users_csv.describe()
display(user)

print("\n_____")
print("\nUsers Table First Few Rows")
print("_____")

#Print first few rows of users dataframe to understand the data structure.
display(users_csv.head())

#Defining Load_json_files function to load json files.
def Load_json_files(File_path):
    # Read json file using read_json function and resulting DataFrame is being stored
    data = pd.read_json(File_path, lines=True)

    # Replace empty strings with NaN so that we can count empty values.
    data.replace('', np.nan, inplace=True)

    # Check for empty values in all columns and store it in data variable.
    data.isnull().sum()

    return data

#Load json files into products_json variable by calling Load_json_files function and print
products_json = Load_json_files('jcpenney_products.json')
print("\n\n")
print("_____")
print("\njcpenney_products.json")
print("_____")
display(products_json)

print("\n")
print("_____")
print("\nproducts_json Table Statistics")
print("_____ \n")
display(products_json.info())

print("\n")
print("_____")
print("\nproducts_json Table Description")
print("_____ \n")
display(products_json.describe())

#Load json files into reviews_json variable by calling Load_json_files function and print
reviews_json = Load_json_files('jcpenney_reviewers.json')
print("\n\n")
print("_____")
print("\njcpenney_reviewers.json")
print("_____")
display(reviews_json)

```

```

print("\n")
print("_____")
print("\nreviews_json Table Statistics")
print("_____ \n")
display(reviews_json.info())

print("\n")
print("_____")
print("\nreviews_json Table description")
print("_____ \n")
display(reviews_json.describe())

```

Product.csv Table Description

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7982 entries, 0 to 7981
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Uniq_id         7982 non-null   object
1   SKU             7915 non-null   object
2   Name            7982 non-null   object
3   Description      7439 non-null   object
4   Price           5816 non-null   float64
5   Av_Score        7982 non-null   float64
dtypes: float64(2), object(4)
memory usage: 374.3+ KB
None

```

Product.csv Table Statistics

	Price	Av_Score
count	5816.000000	7982.000000
mean	144.776618	2.988683
std	499.223719	0.911673
min	-65.270000	1.000000
25%	40.700000	2.500000
50%	58.010000	3.000000
75%	87.020000	3.500000
max	17122.170000	5.000000

Product Table First Few Rows

	Uniq_id	SKU	Name	Description	Price	Av_Score
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capr...	41.09	2.625
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capr...	41.09	3.000
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capr...	41.09	2.625
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capr...	41.09	3.500
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capr...	41.09	3.125

Reviews.csv Table Description

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39063 entries, 0 to 39062
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Uniq_id     39063 non-null  object
1   Username    39063 non-null  object
2   Score       39063 non-null  int64
3   Review      39063 non-null  object
dtypes: int64(1), object(3)
memory usage: 1.2+ MB
None
```

Reviews.csv Table Statistics

Score	
count	39063.000000
mean	1.487648
std	1.400332
min	0.000000
25%	0.000000
50%	1.000000
75%	2.000000
max	5.000000

Reviews Table First Few Rows

	Uniq_id	Username	Score	Review
0	b6c0b6bea69c722939585baeac73c13d	fsdv4141	2	You never have to worry about the fit...Alfred...
1	b6c0b6bea69c722939585baeac73c13d	krpz1113	1	Good quality fabric. Perfect fit. Washed very ...
2	b6c0b6bea69c722939585baeac73c13d	mbmg3241	2	I do not normally wear pants or capris that ha...
3	b6c0b6bea69c722939585baeac73c13d	zeqq1222	0	I love these capris! They fit true to size and...
4	b6c0b6bea69c722939585baeac73c13d	nvfn3212	3	This product is very comfortable and the fabri...

Users.csv Table Description

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Username    5000 non-null   object
1   DOB         5000 non-null   object
2   State       5000 non-null   object
dtypes: object(3)
memory usage: 117.3+ KB
None
```

Users.csv Table Statistics

	Username	DOB	State
count	5000	5000	5000
unique	4999	52	57
top	dqft3311	07.08.1953	Massachusetts
freq	2	112	107

Users Table First Few Rows

	Username	DOB	State
0	bkpn1412	31.07.1983	Oregon
1	gqjs4414	27.07.1998	Massachusetts
2	eehe1434	08.08.1950	Idaho
3	hkxj1334	03.08.1969	Florida
4	jjbd1412	26.07.2001	Georgia

jcpenny_products.json

	uniq_id	sku	name_title	description	list_price	sale_pri
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	24
1	93e5272c51d8cce02597e3ce67b7ad0a	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	24
2	013e320f2f2ec0cf5b3ff5418d688528	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	24
3	505e6633d81f2cb7400c0cfa0394c427	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	24
4	d969a8542122e1331e304b09f81a83f6	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on cap...	41.09	24
...	...	...	...	...	...	...
7977	16e3ca6b6a2d3f5da9a81df6cfe7e27a	pp5002691549	Hoover® WindTunnel® Upright Vacuum Cleaner	This Hoover® vacuum features dual-stage cyclon...	201.97	130
7978	209bec04d9f194616358a77d7b41b314	pp5002691549	Hoover® WindTunnel® Upright Vacuum Cleaner	This Hoover® vacuum features dual-stage cyclon...	201.97	130
7979	7a8a7cba7b69b4c46ceac1d9666d84f0	pp5002691549	Hoover® WindTunnel® Upright Vacuum Cleaner	This Hoover® vacuum features dual-stage cyclon...	201.97	130
7980	9887bb20c12be3c09d094e97ac8d22be	pp5005921226	Hope Chest Embroidered	NaN	NaN	35.631

	uniq_id	sku	name_title	description	list_price	sale_price
			Quilt & Accessories			
7981	2cc49292b44cc12fe22206440d3e7472	pp5005921226	Hope Chest Embroidered Quilt & Accessories	NaN	NaN	35.631

7982 rows × 15 columns

products_json Table Statistics

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7982 entries, 0 to 7981
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   uniq_id                               7982 non-null   object
1   sku                                   7915 non-null   object
2   name_title                           7982 non-null   object
3   description                           7439 non-null   object
4   list_price                           5816 non-null   object
5   sale_price                           7964 non-null   object
6   category                             7346 non-null   object
7   category_tree                        7346 non-null   object
8   average_product_rating               7982 non-null   float64
9   product_url                          7982 non-null   object
10  product_image_urls                   7825 non-null   object
11  brand                               7982 non-null   object
12  total_number_reviews                 7982 non-null   int64
13  Reviews                             7982 non-null   object
14  Bought With                          7982 non-null   object
dtypes: float64(1), int64(1), object(13)
memory usage: 935.5+ KB
None
```

products_json Table Description

	average_product_rating	total_number_reviews
count	7982.000000	7982.000000
mean	2.988683	4.893886
std	0.911673	3.314284
min	1.000000	1.000000
25%	2.500000	2.000000
50%	3.000000	4.000000
75%	3.500000	8.000000
max	5.000000	23.000000

jcpenny_reviewers.json

	Username	DOB	State	Reviewed
0	bkpn1412	31.07.1983	Oregon	[cea76118f6a9110a893de2b7654319c0]
1	gqjs4414	27.07.1998	Massachusetts	[fa04fe6c0dd5189f54fe600838da43d3]
2	eehe1434	08.08.1950	Idaho	[]
3	hkxj1334	03.08.1969	Florida	[f129b1803f447c2b1ce43508fb822810, 3b0c9bc0be6...
4	jjbd1412	26.07.2001	Georgia	[]
...	...	...	...	...
4995	mfnn1212	27.07.1997	Delaware	[d6cd506246bd17afa611b6a06236713c]
4996	ejnb3414	01.08.1976	Minnesota	[97de1506cd0bcbe50f2797cd0588eb81]
4997	pdzw1433	28.07.1994	Ohio	[799d62906019d910fa744987da184ae7, b8f5deb7b02...
4998	npha1342	07.08.1953	Montana	[6250b1d691cd3842f05b87736f2fadbf]
4999	yuoc2324	02.08.1975	South Dakota	[]

5000 rows × 4 columns

reviews_json Table Statistics

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Username    5000 non-null   object
1   DOB         5000 non-null   object
2   State       5000 non-null   object
3   Reviewed    5000 non-null   object
dtypes: object(4)
memory usage: 156.4+ KB
None
```

reviews_json Table description

	Username	DOB	State	Reviewed
count	5000	5000	5000	5000
unique	4999	52	57	4030
top	dqft3311	07.08.1953	Massachusetts	[]
freq	2	112	107	971

Data Validation

Text Narration of Data Validation

The provided code segment is a data cleaning process on DataFrames products_json, reviews_json, products_csv, users_csv and reviews_csv. The code eliminates missing (null) and negative values from these DataFrames. To manage this cleaning procedure, a function called remove_negative_and_null_values is defined. Using remove_negative_and_null_values, all of the available data is processed. The function recognises numerical columns and substitutes NaN for negative values. The cleaned DataFrame is saved in the cleaned_data variable, and empty rows are removed. The info() method is used to display the cleaned DataFrame information. This code's main objective is to ensure that there are no null or negative values in the DataFrames, giving analysts a clean, consistent dataset.

In [103...

```
#As per data exploration we have observed columns with empty rows and negative values.
#function to drop and remove empty and negative values to get consistent data.
```

```

# Defined funtion to remove negative and null values from the available data.
def remove_negative_and_null_values(dataframe):
    # Identify columns with numerical values.
    numeric_columns = dataframe.select_dtypes(include='number')

    # Replace negative values with NaN.
    dataframe[numeric_columns.columns] = dataframe[numeric_columns.columns].apply(lambda x: x if x >= 0 else None)

    #drop empty values and store data in cleaned_data variable.
    cleaned_data = dataframe.dropna()

    #return clean_data in function response.
    return cleaned_data

print("\n_____")
print("\nData after removal of negative and Missing values in products_json:")
print("_____")

#Function call to remove negative and null values from product json file
products_json = remove_negative_and_null_values(products_json)

#print product.json file info after data cleaning
display(products_json.info())

print("\n_____")
print("\nData after removal of negative and Missing values in reviews_json:")
print("_____")

#Function call to remove negative and null values from review json file
reviews_json = remove_negative_and_null_values(reviews_json)

#print Review.json file info after data cleaning
display(reviews_json.info())

print("\n_____")
print("\nData after removal of negative and Missing values in products_csv:")
print("_____")

#Function call to remove negative and null values from product csv file
products_csv = remove_negative_and_null_values(products_csv)

#print Product.csv file info after data cleaning
display(products_csv.info())

print("\n_____")
print("\nData after removal of negative and Missing values in users_csv:")
print("_____")

#Function call to remove negative and null values from user csv file
users_csv = remove_negative_and_null_values(users_csv)

#print users.csv file info after data cleaning

```

```

display(users_csv.info())

print("\n_____")
print("\nData after removal of negative and Missing values in reviews_csv:")
print("_____")

#Function call to remove negative and null values from review json file
reviews_csv = remove_negative_and_null_values(reviews_csv)

#print reviews.csv file info after data cleaning
display(reviews_csv.info())

```

Data after removal of negative and Missing values in products_json:

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 5246 entries, 0 to 7979
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   uniq_id               5246 non-null  object
1   sku                   5246 non-null  object
2   name_title            5246 non-null  object
3   description            5246 non-null  object
4   list_price            5246 non-null  object
5   sale_price            5246 non-null  object
6   category              5246 non-null  object
7   category_tree         5246 non-null  object
8   average_product_rating 5246 non-null  float64
9   product_url           5246 non-null  object
10  product_image_urls     5246 non-null  object
11  brand                 5246 non-null  object
12  total_number_reviews   5246 non-null  int64
13  Reviews               5246 non-null  object
14  Bought With           5246 non-null  object
dtypes: float64(1), int64(1), object(13)
memory usage: 655.8+ KB
None

```

Data after removal of negative and Missing values in reviews_json:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Username    5000 non-null  object
1   DOB         5000 non-null  object
2   State       5000 non-null  object
3   Reviewed    5000 non-null  object
dtypes: object(4)
memory usage: 156.4+ KB
None

```

Data after removal of negative and Missing values in products_csv:

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 5723 entries, 0 to 7979
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Uniq_id     5723 non-null   object
1   SKU         5723 non-null   object
2   Name        5723 non-null   object
3   Description  5723 non-null   object
4   Price       5723 non-null   float64
5   Av_Score    5723 non-null   float64
dtypes: float64(2), object(4)
memory usage: 313.0+ KB
None
```

Data after removal of negative and Missing values in users_csv:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Username    5000 non-null   object
1   DOB         5000 non-null   object
2   State       5000 non-null   object
dtypes: object(3)
memory usage: 117.3+ KB
None
```

Data after removal of negative and Missing values in reviews_csv:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39063 entries, 0 to 39062
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Uniq_id     39063 non-null   object
1   Username    39063 non-null   object
2   Score       39063 non-null   int64
3   Review      39063 non-null   object
dtypes: int64(1), object(3)
memory usage: 1.2+ MB
None
```

Data Visualization

Text Narration of Data Visualization

This code performs exploratory data analysis on a dataset containing information about users and their states. Extracting and visualizing the overall distribution of users across different states using a pie chart. It uses the 'State' column from the 'users_csv' DataFrame and counts the number of users in each state.

Distribution of Users in Bottom Five States:

It identifies the top five states with the highest number of customers. It creates a bar chart to display the users in these top five states. The order of states on the x-axis is determined by the count of users.

Distribution of Users in Bottom Five States:

Identifies the five states with the least number of customers using the `nsmallest` function. Another bar chart is created to visualize the distribution of users in these bottom five states. These visualizations provide distribution of users, highlighting states with the highest and lowest user counts.

In [109...

```
#Overall Distribution of Users by State

# Store overall user count in state_counts variable extracted from user_csv table.
state_counts = users_csv['State'].value_counts()

# Pie chart for data presentation.
plt.figure(figsize=(25, 18))
plt.pie(state_counts, labels=state_counts.index, autopct='%1.1f%%', startangle=140)

# To make a circular chart set expect ration to equal.
plt.axis('equal')

# Title of chart
plt.title('Distribution of Users Overall by State\n\n')
print("\n\n")
plt.show()

# Get the top five states by number of customers
top_five_states = users_csv['State'].value_counts().nlargest(5).index

# Applying filter to get top five states
users_top_five_states = users_csv[users_csv['State'].isin(top_five_states)]

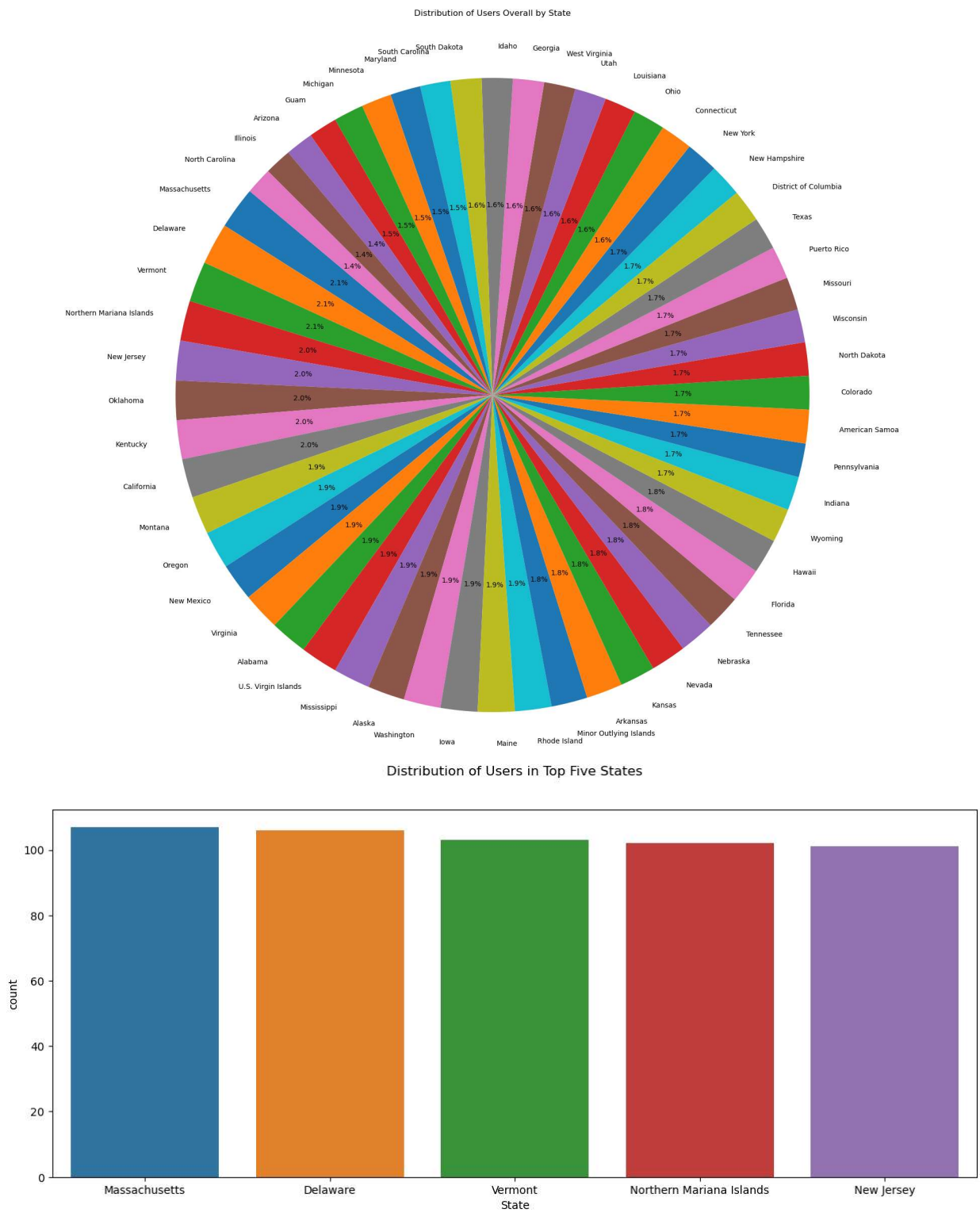
# Plot the extracted count of top states
plt.figure(figsize=(15, 6))
sns.countplot(x='State', data=users_top_five_states, order=top_five_states)
plt.title('Distribution of Users in Top Five States\n\n')
plt.show()

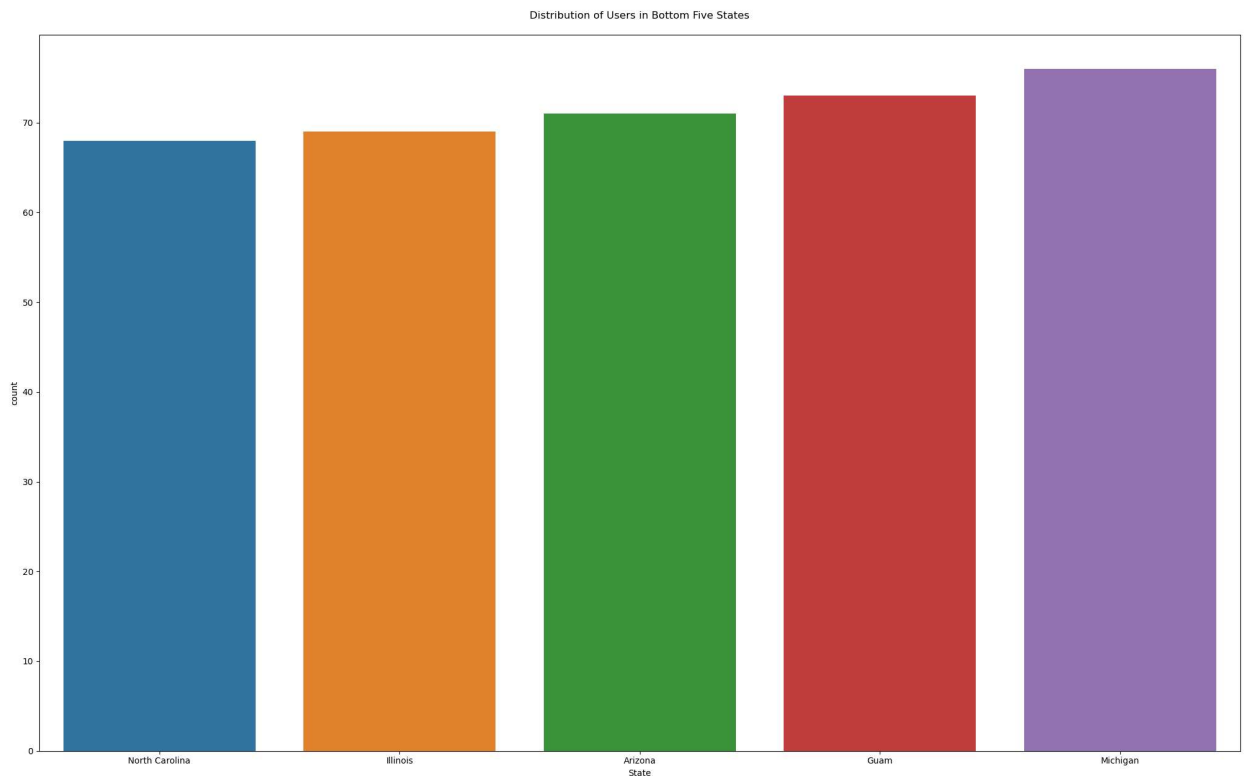
# Get the five states with the Least number of customers using nsmallest function
top_five_states = users_csv['State'].value_counts().nsmallest(5).index

# Filter the DataFrame for the bottom five states
users_top_five_states = users_csv[users_csv['State'].isin(top_five_states)]

# Plot the count for the bottom five states
```

```
plt.figure(figsize=(25, 15))
sns.countplot(x='State', data=users_top_five_states, order=top_five_states)
plt.title('Distribution of Users in Bottom Five States\n')
plt.show()
```





Text Narration

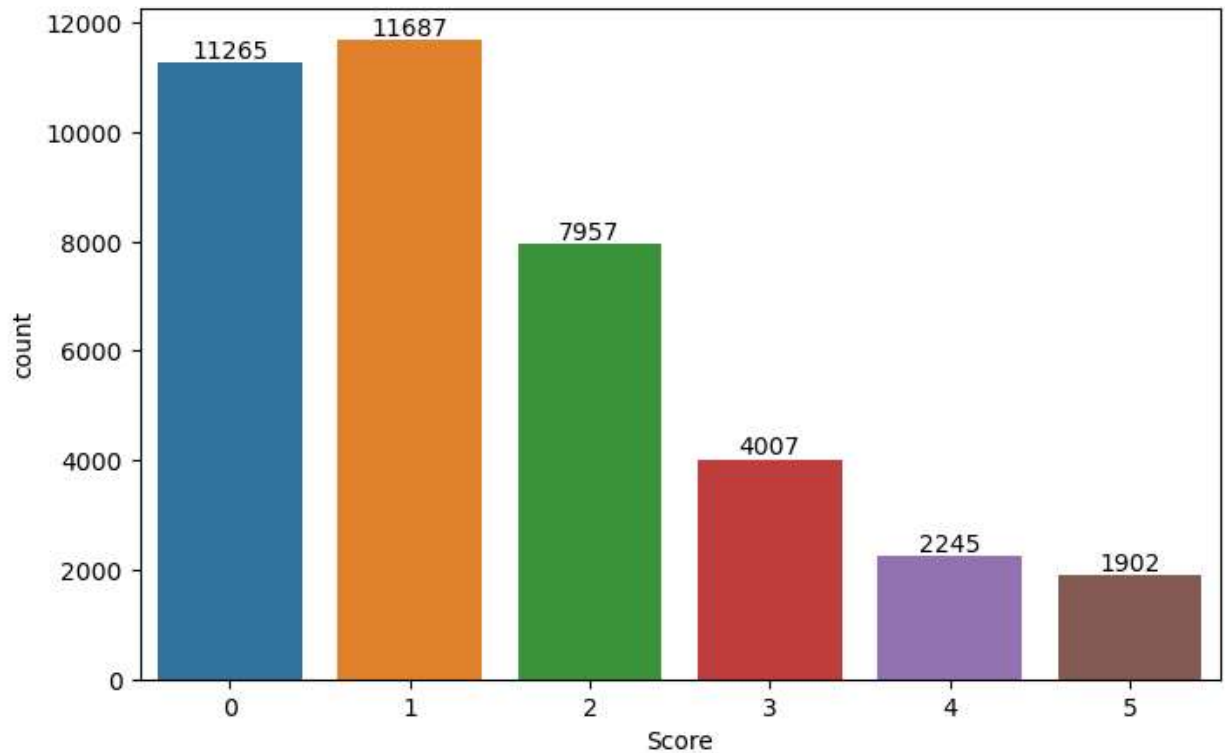
The distribution of review scores is represented by a bar chart that is created by the code using Seaborn's countplot. The potential review scores are shown on the x-axis, and the number of reviews for each score is shown on the y-axis. Each bar now has annotations at the top showing the precise number of reviews for that specific score. The distribution of review scores is summarised in the chart, making it easy to quickly determine how frequently each score occurs.

```
In [110... # Plot the distribution of review scores from review.csv file
plt.figure(figsize=(8, 5))
ax = sns.countplot(x='Score', data=reviews_csv)

# Add count on top of the bars
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height() + 5),
                ha='center', va='center', fontsize=10, color='black', xytext=(0, 5),
                textcoords='offset points')

plt.title('Distribution of Review Scores\n\n')
plt.show()
```

Distribution of Review Scores



Text Narration

The provided code is focused on visualizing the top categories based on the 'average_product_rating' in a dataset. A bar chart is created using Seaborn's barplot to visually represent the selected top categories and their corresponding average product ratings. The categories are displayed on the y-axis, while the x-axis represents the average product ratings. The bars are sorted in descending order based on the average product ratings. Data labels indicating the exact average product ratings are added on top of each bar for clarity. The resulting chart provides a clear overview of the top categories and their average product ratings in a descending order.

```
In [112... # Remove duplicates in the 'category' column
df_unique_categories = products_json.drop_duplicates(subset=['category'])

# Select the top categories to display
top_n = 20
df_top_categories = df_unique_categories.sort_values(by='average_product_rating', asce

# Width and height of the chart
plt.figure(figsize=(12, 8))

# Create a bar chart
barplot = sns.barplot(x='average_product_rating', y='category', data=df_top_categories

# Add data labels on top of the bars
for p in barplot.patches:
```

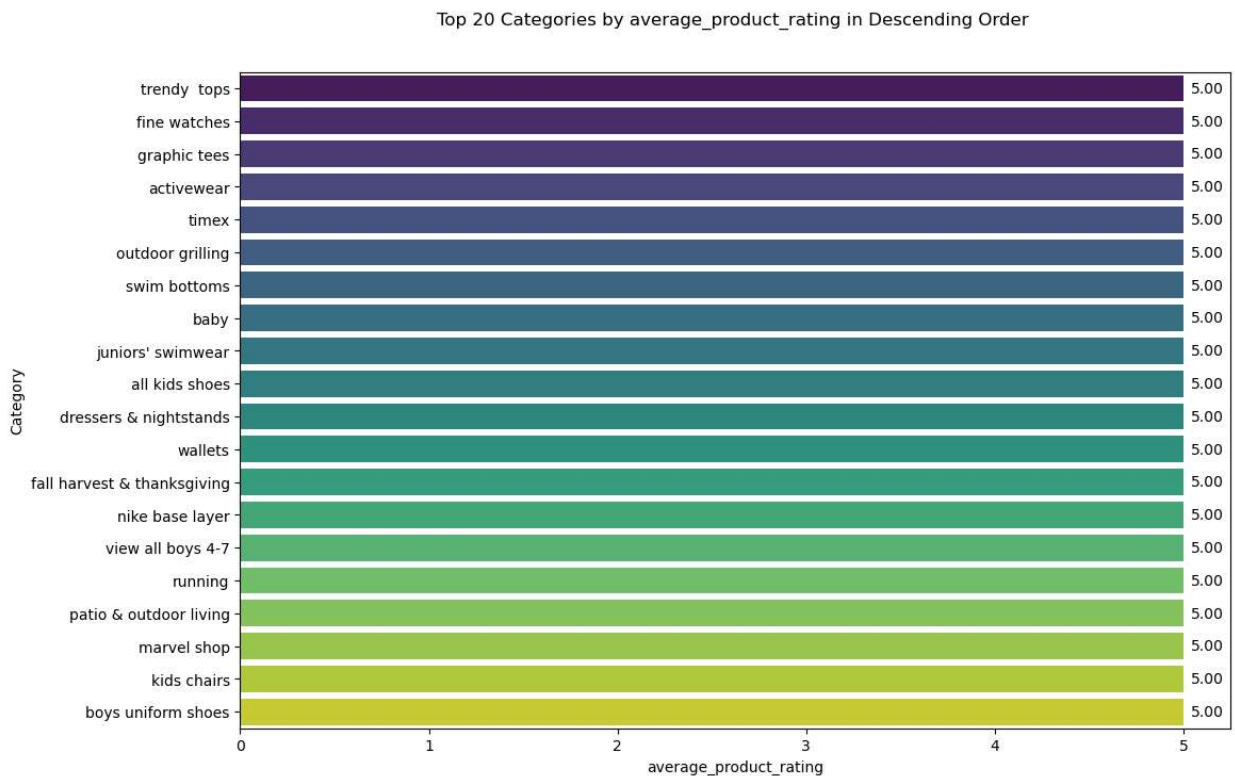
```

    barplot.annotate(f'{p.get_width():.2f}', (p.get_x() + p.get_width(), p.get_y() + p
                    xytext=(5, 0), textcoords='offset points', ha='left', va='center')

plt.title(f'Top {top_n} Categories by average_product_rating in Descending Order \n\n')
plt.xlabel('average_product_rating')
plt.ylabel('Category')

plt.show()

```



Text Narration

The 'DOB' (Date of Birth) column in the DataFrame is converted to datetime format using the `pd.to_datetime` function. The specified format is '%d.%m.%Y', indicating the day, month, and year format. The age of each user is calculated by subtracting their date of birth from the current date (today). The result is stored in a new 'Age' column in the DataFrame. A histogram is created to visualize the distribution of user ages. The ages are grouped into bins (20 bins in this case), and the frequency of users in each age group is represented by the height of the bars.

```

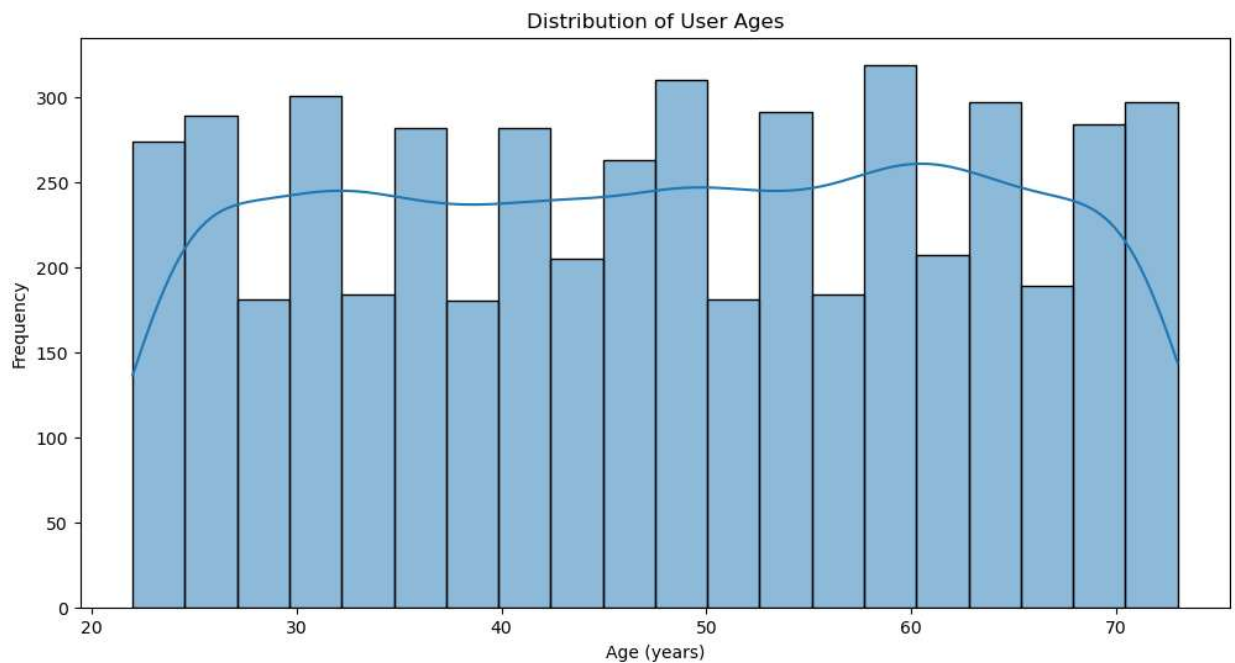
In [113... # Convert DOB to datetime and calculate age
users_csv['DOB'] = pd.to_datetime(users_csv['DOB'], format='%d.%m.%Y')
users_csv['Age'] = (pd.to_datetime('today') - users_csv['DOB']).astype('<m8[Y]')

# Plot the histogram of user ages
plt.figure(figsize=(12, 6))

# extract data from user_csv file from DOB
sns.histplot(users_csv['Age'], bins=20, kde=True)
plt.title('Distribution of User Ages')
plt.xlabel('Age (years)')

```

```
plt.ylabel('Frequency')
plt.show()
```



Data Analysis

Text Narration of Data Analysis

It merges the 'reviews_csv' and 'users_csv' DataFrames using the 'Username' column to create a merged DataFrame named 'UserReview_merged_data.' It groups the merged DataFrame by 'State' and counts the number of reviews for each state, creating a new DataFrame named 'reviews_per_state.' It sorts the 'reviews_per_state' DataFrame in descending order based on the number of reviews. It identifies the state with the highest number of reviews and stores the relevant information in the 'state_with_highest_reviews' variable. Data Visualization - Bar Chart: It creates a bar chart to visually represent the number of reviews per state. Each bar represents a state, and the height of the bar corresponds to the number of reviews. The x-axis shows the states, and the y-axis shows the number of reviews.

```
In [114... #We are going to check which state has highest number of reviews by merging user.csv a

# Merging reviews.csv and users.csv to create merged table by username column
UserReview_merged_data = pd.merge(reviews_csv, users_csv, on='Username')

display(UserReview_merged_data)

# Group by 'State' and count the number of reviews for each state
reviews_per_state = UserReview_merged_data.groupby('State')['Review'].count().reset_in

# Sort the DataFrame by the number of reviews in descending order
reviews_per_state = reviews_per_state.sort_values(by='Review', ascending=False)
```

```

# Find the state with the highest number of reviews
state_with_highest_reviews = reviews_per_state.iloc[0] # Change from loc to iloc for

# Data Visualization Example:
plt.figure(figsize=(17, 8)) # Adjust the width and height as needed

# Create a bar chart
plt.bar(reviews_per_state['State'], reviews_per_state['Review'], color='skyblue')

plt.title('Number of Reviews Per State\n\n\n')
plt.xlabel('State')
plt.ylabel('Number of Reviews')

plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability

# Highlight the state with the highest number of reviews
plt.annotate(f'Highest: {state_with_highest_reviews["Review"]} reviews',
            xy=(state_with_highest_reviews.name, state_with_highest_reviews['Review'])
            xytext=(state_with_highest_reviews.name, state_with_highest_reviews['Review']
            )

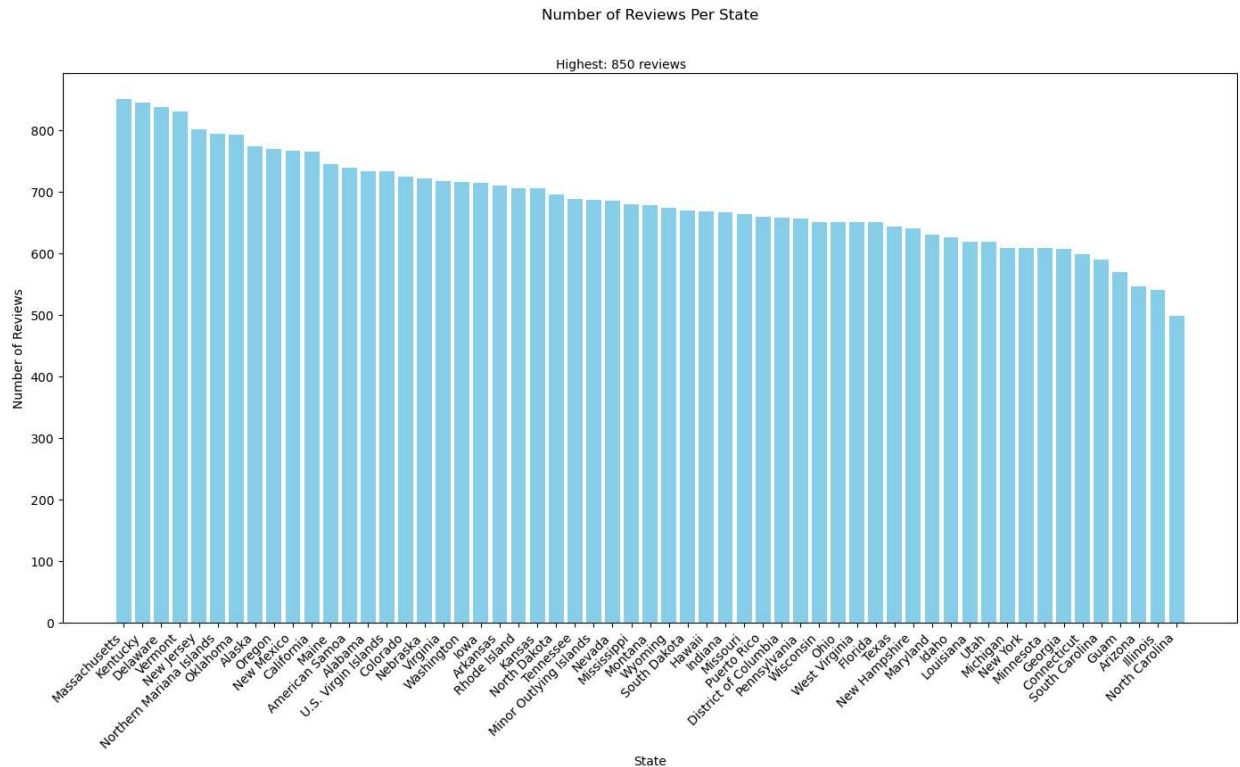
plt.show()

```

	Uniq_id	Username	Score	Review	DOB	State	Age
0	b6c0b6bea69c722939585baeac73c13d	fsdv4141	2	You never have to worry about the fit...Alfred...	1980-07-31	American Samoa	43.0
1	cbe8d131628ec67e803c47d3dd6f2529	fsdv4141	2	These are great shirts, looks great all day. W...	1980-07-31	American Samoa	43.0
2	5ea5f53bbb750106865a044634404dd7	fsdv4141	5	I purchase three rugs to replace twenty-year-o...	1980-07-31	American Samoa	43.0
3	0144d2094668b42ae7c674915806f5f3	fsdv4141	1	I am a huge user of BE original fromula. I hap...	1980-07-31	American Samoa	43.0
4	99141a2b164cf257c96bcb4593915b50	fsdv4141	1	Very soft and stretchy! They arent as dressy a...	1980-07-31	American Samoa	43.0
...	...	...	...	...	...	...	...
39075	307e9f01bb5c6c0fd6b4cee74828c22c	zmon4424	1	Size chart suggested I order size 6, my usual ...	2001-07-26	District of Columbia	22.0
39076	add769ea4635149ad92c0b01fd5d31ac	hnxb3322	2	I bought a pair of Xersion Yoga pants several ...	1976-08-01	Connecticut	47.0
39077	1382f508f14050ae2013dd215ed2b82f	jhof4341	2	Impressed with the detailing - back pleats, st...	1988-07-29	New Hampshire	35.0
39078	f08c827fb5b4b4533bd9ec6e42dedff8	jhof4341	0	Use these for golf slacks. Dont get in my way....	1988-07-29	New Hampshire	35.0

	Uniq_id	Uusername	Score	Review	DOB	State	Age
39079	9aed892fd225eb90ddee0979da5bce84	dddt3113	0	I purchased this based on past Stafford purcha...	1994-07-28	Alabama	29.0

39080 rows × 7 columns



In [120...

```
#Top 10 products by number of reviews

# Merge reviews and products tables by Uniq_id
ProductReview_merged_data = pd.merge(reviews_csv, products_csv, on='Uniq_id')

# Group by 'SKU' and count the number of reviews for each product
reviews_per_product = ProductReview_merged_data.groupby(['SKU', 'Name'])['Review'].count()

# Sort the DataFrame by the number of reviews in descending order
reviews_per_product = reviews_per_product.sort_values(by='Review', ascending=False)

# Select the top 10 products
top_10_products = reviews_per_product.head(15)

# Data Visualization Example:
plt.figure(figsize=(10, 8)) # Adjust the width and height as needed

# Create a bar chart
barplot = sns.barplot(x='Review', y='Name', data=top_10_products, color='lightgreen')

# Add data labels on top of the bars
for p in barplot.patches:
    barplot.annotate(f'{p.get_width():.0f}', (p.get_width(), p.get_y() + p.get_height()),
                    ha='left', va='center', fontsize=10, color='black', xytext=(5, 0))
```

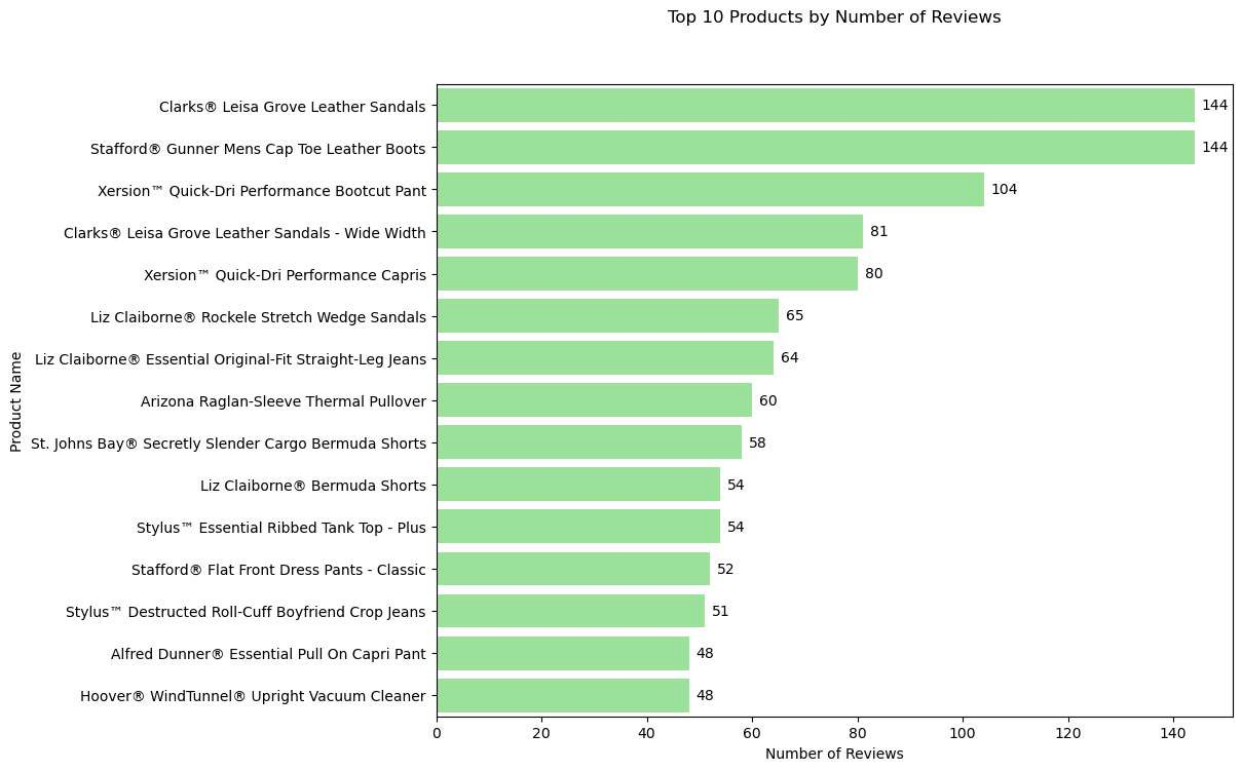
```

textcoords='offset points')

plt.title('Top 10 Products by Number of Reviews\n\n\n')
plt.xlabel('Number of Reviews')
plt.ylabel('Product Name')

plt.show()

```



```

In [116... # Replace the column names accordingly

# Group by 'brand' and count the number of distinct products (using 'sku')
brands_product_count = products_json.groupby('brand')['sku'].nunique().reset_index()

# Find the top 10 brands with the most number of distinct products
top_brands_most_products = brands_product_count.nlargest(10, 'sku')

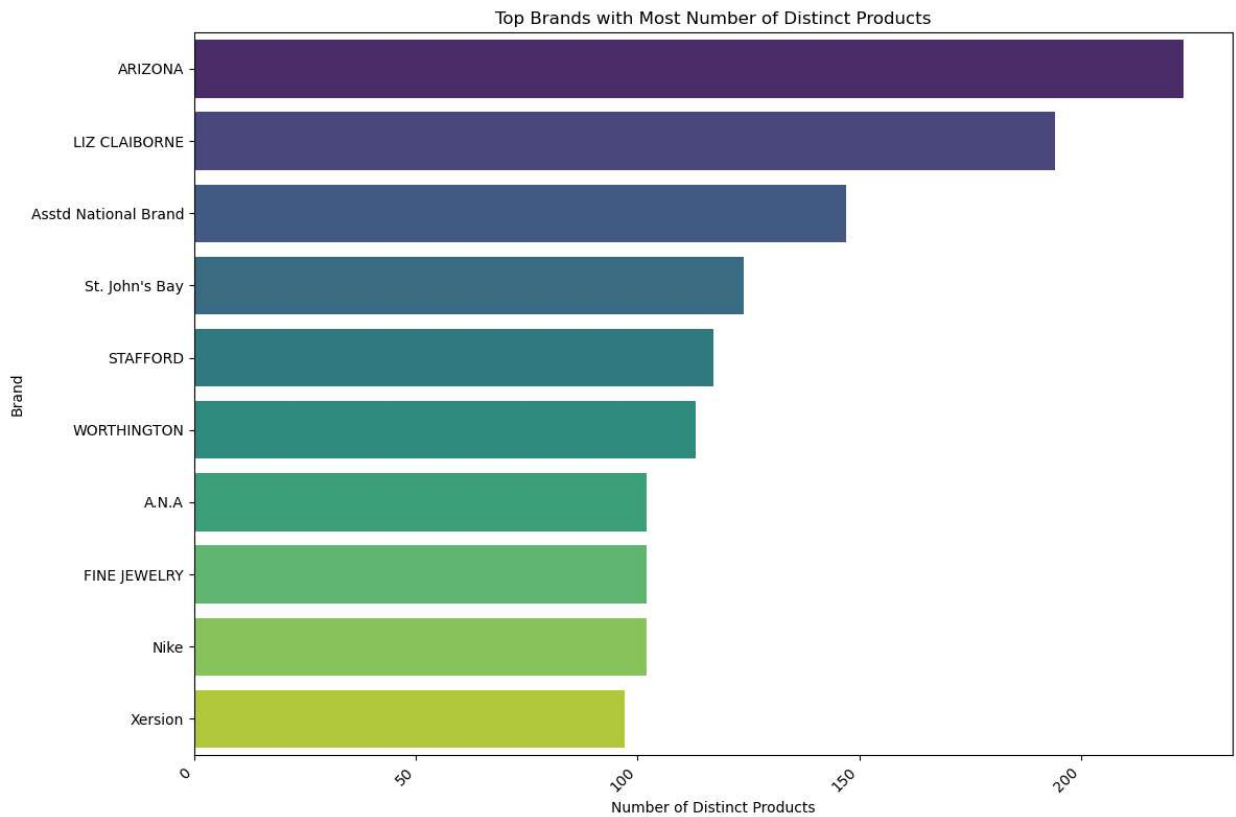
# Data Visualization Example:
plt.figure(figsize=(12, 8)) # Adjust the width and height as needed

# Create a bar chart
sns.barplot(x='sku', y='brand', data=top_brands_most_products, palette='viridis')
plt.title('Top Brands with Most Number of Distinct Products')
plt.xlabel('Number of Distinct Products')
plt.ylabel('Brand')

plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better visibility
plt.tight_layout()

plt.show()

```



```
In [117... # Performing an inner join on 'Uniq_id' column
merged_table = pd.merge(products_csv, reviews_csv, on='Uniq_id', how='inner')

print("_____")
print("\nMerged Table from users.csv, reviews.csv and products.csv")
print("_____")

# Performing an inner join on 'Username' column
combined_table = pd.merge(merged_table, users_csv, on='Username', how='inner')
display(combined_table)

# Filter data for the state of Massachusetts
ma_data = combined_table[combined_table['State'] == 'Massachusetts']

# Find the top 5 products by price
top_5_products = ma_data.nlargest(5, 'Price')

# Display the top 5 products
print("Top 5 Products in Massachusetts by Price:")
display(top_5_products[['Uniq_id', 'Name', 'Description', 'Price']])

# Set the figure size
plt.figure(figsize=(9, 6))

# Create a bar graph
bars = plt.bar(top_5_products['Name'], top_5_products['Price'], color='skyblue')

# Add labels and title
```

```
plt.xlabel('Product Name')
plt.ylabel('Price')
plt.title('Top 5 Products in Massachusetts by Price\n')

# Rotate x-axis labels for better visibility
plt.xticks(rotation=45, ha='right')

# Print values on top of the bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, yval, round(yval, 2), ha='center', va='bottom')

# Display the graph
plt.tight_layout()
plt.show()
```

Merged Table from users.csv, reviews.csv and products.csv

	Uniq_id	SKU	Name	Description	Price	Av_Score
0	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capri...	41.09	2.62!
1	5ea5f53bbb750106865a044634404dd7	pp5006020188	Liz Claiborne® Serenity Bath Rug Collection	Defined by its tufted ogee pattern and calming...	36.26	3.500
2	99141a2b164cf257c96bcb4593915b50	pp5005740454	Zoomers Maternity Knit Gaucho Pants	Keep your maternity style simple and polished ...	36.26	2.200
3	7c27ffd820cbe7041c2a8bd0ca92a232	pp5003071388	Mele & Co. Mens Black Faux Leather Watch Box	A handsome way to store your watches, this bla...	97.84	3.000
4	b6c0b6bea69c722939585baeac73c13d	pp5006380337	Alfred Dunner® Essential Pull On Capri Pant	You'll return to our Alfred Dunner pull-on capri...	41.09	2.62!
...	...	...	...	...	...	...
26526	758cc6190f00768bd098a4ac589db3d7	pp5004710245	Glamorise® Embroidered Wonderwire® Full-Covera...	Beautiful embroidery lends elegance to our com...	57.03	3.12!
26527	217b4a1b3badf0579af3c6d5b4913657	pp5006660007	Stylus™ Linen Crop Pants	Whether you wear them for work or play, our co...	53.18	4.250
26528	15624772ab859474f72d48e753230021	pp5006300356	GE® 1.7 Cu. Ft. Over-the Range Microwave Oven ...	true Touch controls and a scrolling electronic...	337.21	2.37!
26529	0a43b5db2ac654f9fb365a123a9e81e8	pp5006462153	Ninety Six Degrees Embroidered Triangle One-Pi...	This one-piece comes to life with the gorgeous...	100.99	2.500

	Uniq_id	SKU	Name	Description	Price	Av_Score
26530	9aed892fd225eb90ddee0979da5bce84	pp5006153172	Stafford® Travel Short-Sleeve Easy-Care Broadc...	This Stafford everyday dress shirt is a wardro...	54.02	3.500

26531 rows × 12 columns

Top 5 Products in Massachusetts by Price:

	Uniq_id	Name	Description	Price
9619	9633993c92db844e5bf57865640dd5d1	GE Profile™ 30 Built-In Double Wall Oven Self ...	true Be able to cook the way you need to with ...	4470.70
12052	6f41b7a770070d43581a6a6d54c5da65	GE Café™ 27 Built-In Single Wall Oven With Co...	true The true European convection in this oven...	3020.35
2175	f780d84a47d56c4d5f7ae02985a1698f	GE Profile™ 30 Electric Wall Oven Self Cleanin...	true Cooking is a breeze with this true Europe...	2778.63
4238	a229f8478997363b3c10dbb6e400dc7e	GE® 30 Free-Standing Double Oven Gas Range wit...	true GE®s free-standing range with double oven...	2053.45
12627	6b4338d361e5ba6ad378c8c0ccbabb19	GE® ENERGY STAR® 21.8 Cu. Ft. 33.5 Wide Side-B...	true This spacious side-by-side refrigerator c...	1811.73

Top 5 Products in Massachusetts by Price

